Solution: Minimum Number of Vertices to Reach All Nodes(medium)

Problem Statement

Given a directed acyclic graph with `n` nodes labeled from `0` to `n-1`, determine the smallest number of initial nodes such that you can access all the nodes by traversing edges. Return these nodes.

Examples

1. **Input**:
   - `n = 6`
   - `edges = [[0,1],[0,2],[2,5],[3,4],[4,2]]`

   Expected Output: `[0,3]`

   Justification: Starting from nodes 0 and 3, you can reach all other nodes in the graph. Starting from node 0, you can reach nodes 1, 2, and 5. Starting from node 3, you can reach nodes 4 and 2 (and by extension 5).

2. **Input**:
   - `n = 3`
   - `edges = [[0,1],[2,1]]`

   Expected Output: `[0,2]`

   Justification: Nodes 0 and 2 are the only nodes that don't have incoming edges. Hence, you need to start from these nodes to reach node 1.

3. **Input**:
   - `n = 5`
   - `edges = [[0,1],[2,1],[3,4]]`

   Expected Output: `[0,2,3]`

   Justification: Node 1 can be reached from both nodes 0 and 2, but to cover all nodes, you also need to start from node 3.

Solution

To solve the problem of determining the minimum number of vertices needed to reach all nodes in a directed graph, we focus on the concept of "in-degree" which represents the number of incoming edges to a node. In a directed graph, if a node doesn't have any incoming edges (in-degree of 0), then it means that the node cannot be reached from any other node. Hence, such nodes are mandatory starting points to ensure that every node in the graph can be reached. Our algorithm thus identifies all nodes with an in-degree of 0 as they are potential starting points to traverse the entire graph.

Steps:

1. **Graph Representation**: Begin by representing the graph using an adjacency list or a similar data structure.
2. **In-Degree Calculation**: Compute the in-degree for all the nodes. The in-degree of a node is the number of edges coming into it. This can be done by initializing an array to keep track of in-degrees for each node and iterating over the edges to update the in-degree counts.
3. **Result Gathering**: Iterate over the computed in-degrees. Nodes with an in-degree of 0 don't have any incoming edges and thus are part of our result set as they serve as starting points.

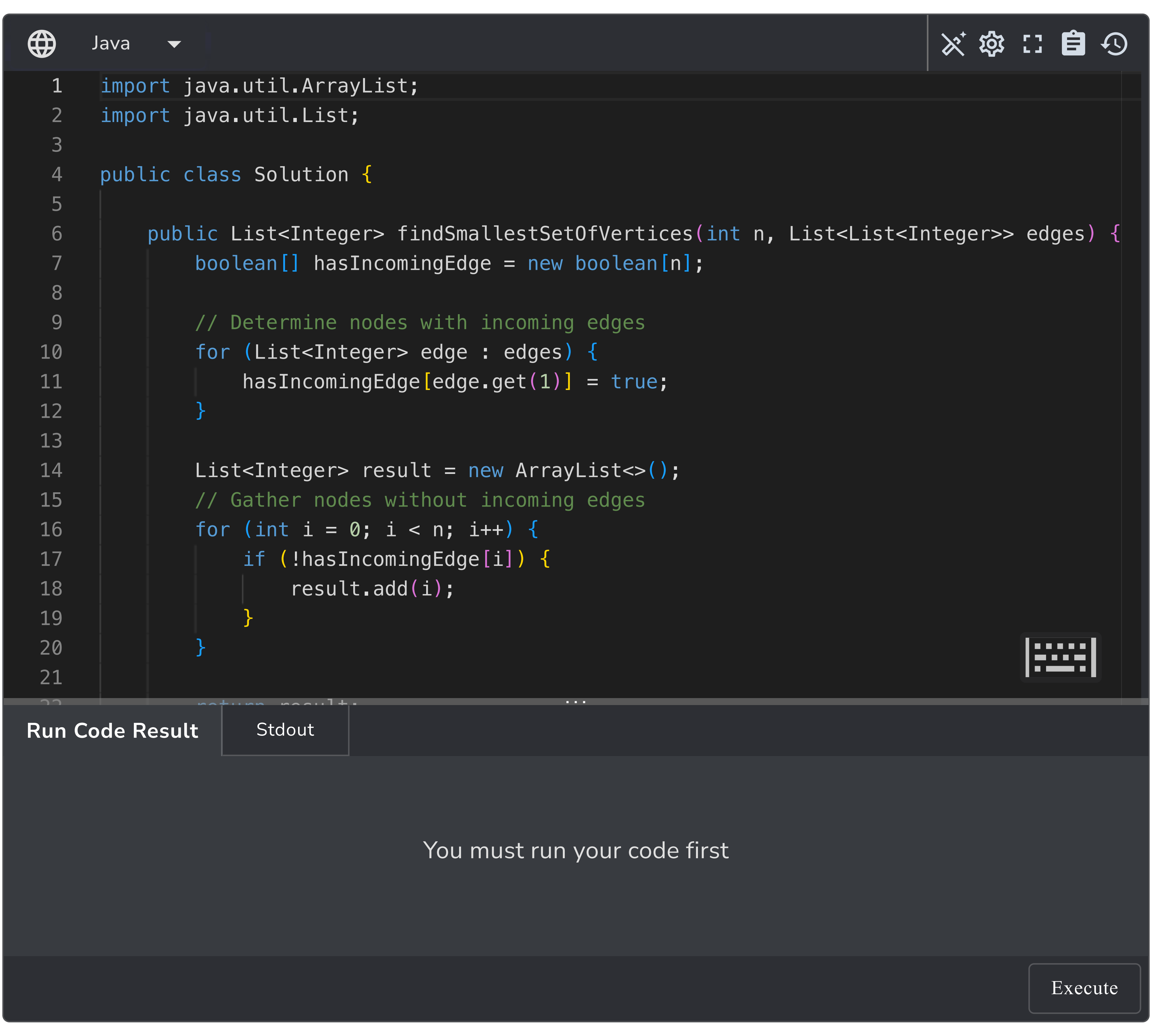
Algorithm Walkthrough:

Let's walk through the solution using the example `n = 6` and `edges = [[0,1],[0,2],[2,5],[3,4],[4,2]]`:

- **Step 1**: Start with an in-degree array initialized to zeros for all nodes: `[0, 0, 0, 0, 0, 0]`.
- **Step 2**: Iterate over the edges. For each edge `(source, destination)`, increment the in-degree of the `destination` node by 1. After processing all edges, the in-degree array becomes: `[0,1,2,0,1,1]`.
- **Step 3**: Finally, by examining the in-degree array, nodes 0 and 3 are identified as having an in-degree of 0. This means they don't receive any incoming edges, and thus, they become our result set: `[0,3]`. Starting from these nodes, we can reach all other nodes in the graph.

Code

Here is the code for this algorithm:



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {

    public List<Integer> findSmallestSetOfVertices(int n, List<List<Integer>> edges) {
        boolean[] hasIncomingEdge = new boolean[n];

        // Determine nodes with incoming edges
        for (List<Integer> edge : edges) {
            hasIncomingEdge[edge.get(1)] = true;
        }

        List<Integer> result = new ArrayList<>();
        // Gather nodes without incoming edges
        for (int i = 0; i < n; i++) {
            if (!hasIncomingEdge[i]) {
                result.add(i);
            }
        }

```

You must run your code first

Time Complexity:

1. **Initialization:** Initializing the set for all nodes in the graph takes (O(n)), where (n) is the number of nodes.
2. **Processing Edges:** For every edge ([u, v]), we are simply checking and potentially removing the node (v) from our set. Since we do a constant amount of work for each edge, this step takes (O(e)) time, where (e) is the number of edges.

Thus, the overall time complexity is the sum of the above two steps, i.e., (O(n + e)). In the worst case (a complete graph), every node is connected to every other node, making (e = n^2), leading to a worst-case time complexity of (O(n^2)). However, this is not a typical scenario, and in most real-world graphs, (e) is often linear or close to linear with respect to (n). Therefore, (O(n + e)) is a more informative measure.

Space Complexity:

1. **Set for Nodes:** The set that we're using to keep track of nodes which do not have any incoming edges will, at most, contain all nodes. This gives us a space complexity of (O(n)).
2. **Graph Representation:** Though we are given the edges as an input, if we consider the space used by this representation, it will be (O(e)) for the edges.

However, note that since we are not using any additional data structures that scale with the size of the graph other than the set for nodes, our primary concern is the set's space. Thus, the dominant term here is (O(n)).

So, the overall space complexity is (O(n)).

In summary:

- Time Complexity: (O(n + e))
- Space Complexity: (O(n))

← Previous
Problem 3: Minimum Number Of Vertices To Reach All Nodes(Medium)

Next →
Introduction To Island Pattern

☐ Mark as Completed